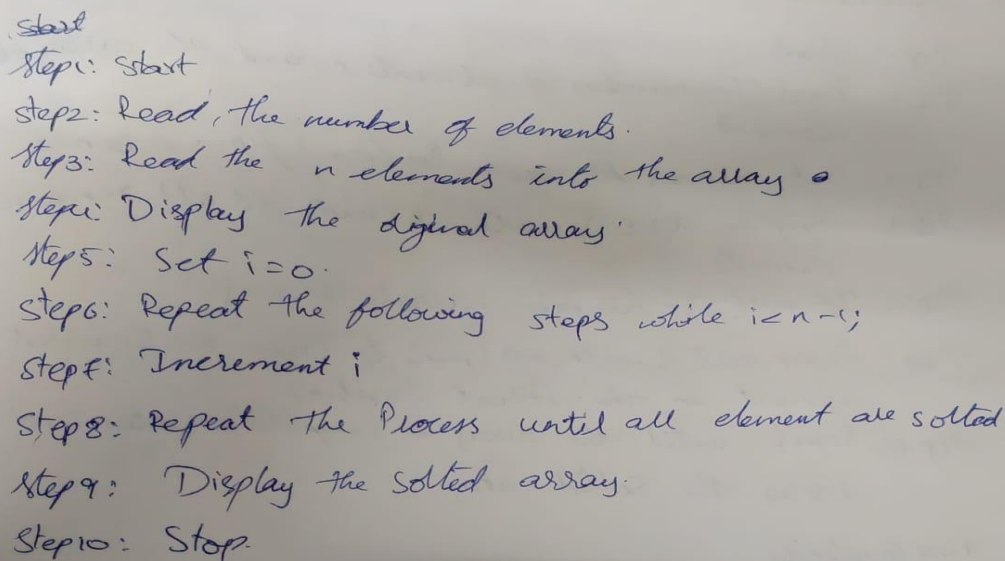


Practical - 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1) Bubble Sort

Algorithm:



start
Step1: Start
Step2: Read the number of elements.
Step3: Read the n elements into the array.
Step4: Display the original array.
Step5: Set $i = 0$.
Step6: Repeat the following steps while $i < n - 1$.
Step7: Increment i .
Step8: Repeat the process until all elements are sorted.
Step9: Display the sorted array.
Step10: Stop.

Program:

```
#include <iostream>
using namespace std;
class BubbleSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
    }
};
```

Name: Yaswanth Sai
Enrollment: 92400118993

```
        cout << endl;
    }
    void logic(){
        int temp;
        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - i - 1; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
};
int main()
{
    BubbleSort bs;
    bs.arrayInput();
    cout << "\nBefore Sorting: ";
    bs.printArray();
    bs.logic();
    cout << "After Sorting: ";
    bs.printArray();
    return 0;
}
```

Output:

```
Output
Enter number of elements: 5
Enter the sorted elements: 3
4
7
6
2
Enter the element to search: 4

Array: 3 4 7 6 2
Element found at index 1
```

Time Complexity:

Time Complexity.

```

for (i=0; i < n-1; i++) {
    for (j=0; j < n-1; j++) {
        if (A[j] > A[j+1]) {
            swap(A[j], A[j+1]);
        }
    }
}

```

$T(n) = n \times n = n^2 = O(n^2)$

Best case = $O(n^2)$

worst case =

$$(n-1) + (n-2) + (n-3) + \dots + 1$$

$$T(n) = 1 + 2 + 3 + \dots + (n-1)$$

$$= \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

Average case: $T(n) = O(n^2)$

2) Selection sort:

Algorithm:

⇒ Selection Sort:

- Step 1: Start
- Step 2: Read the number of elements n and the array element.
- Step 3: Set $\text{min} = i$ for each position i from 0 to $n-2$.
- Step 4: Compare $\text{arr}[j]$ with $\text{arr}[\text{min}]$ for all remaining elements.
- Step 5: If $\text{arr}[j] < \text{arr}[\text{min}]$, update $\text{min} = j$.
- Step 6: Swap $\text{arr}[i]$ with $\text{arr}[\text{min}]$ to place the smallest element in the correct position.
- Step 7: Repeat until the array is completely sorted and display the sorted array.

Program:

```
#include <iostream>

using namespace std;

class SelectionSort{
public:
    int arr[100], n;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void logic(){
        int i, j, min, temp;
        for (i = 0; i < n - 1; i++){
            min = i;
            for (j = i + 1; j < n; j++){
                if (arr[j] < arr[min]){
                    min = j;
                }
            }
        }
        Swap(arr[i], arr[min]);
    }
}

void printArray() {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
};  
int main()  
{  
    SelectionSort ss;  
    ss.arrayInput();  
    cout << "Before Sorting: ";  
    ss.printArray();  
    ss.logic();  
    cout << "After Sorting: ";  
    ss.printArray();  
    return 0;  
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\selectionsort.exe"  
Enter number of elements: 6  
Enter the elements: 12  
9  
2  
8  
31  
1  
Before Sorting: 12 9 2 8 31 1  
After Sorting: 1 2 8 9 12 31
```

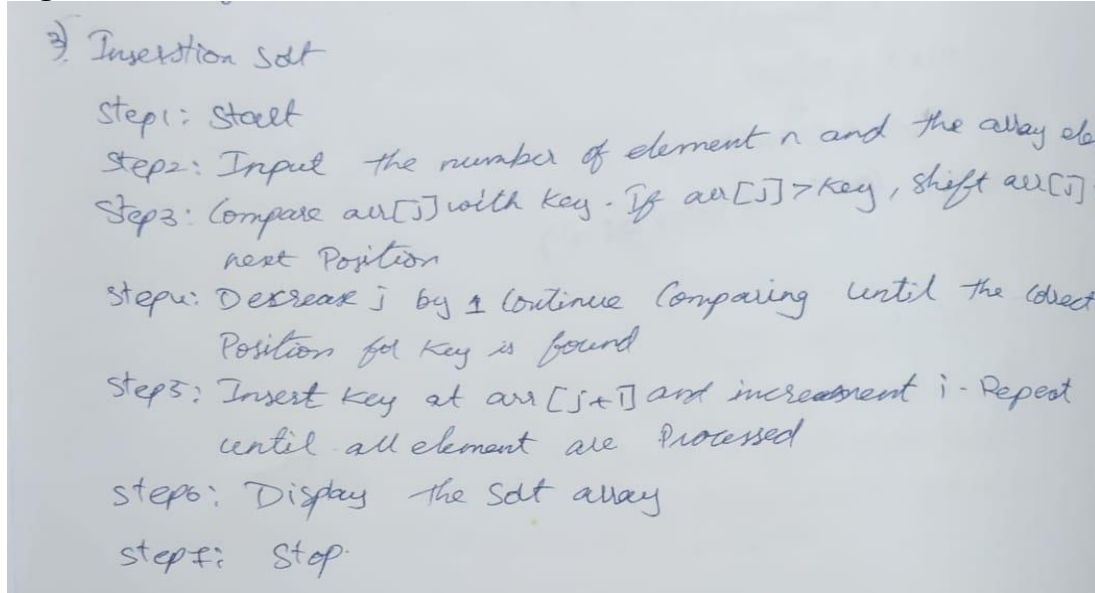
Time complexity:

Time Complexity:

For $(i=0; i \leq n-1; i++) \{ \dots \}$ $\rightarrow n-1$
 $\min = i; \rightarrow C_1$
For $(j=i+1; j \leq n; j++) \{ \dots \}$ $\rightarrow n$
 If $(A[j] < A[\min]) \{ \dots \}$ $\rightarrow C_1$
 $\min = j; \rightarrow C_1$
 Swap $A[i], A[\min]; \rightarrow C_1$
 $\rightarrow$
 $T(n) = (n-1)n$
 $= n^2 - n$
 $T(n) = O(n^2)$
Best case: $O(n^2)$
Worst case: $O(n^2)$
Average case: $O(n^2)$

3) Insertion sort:

Algorithm:



Program:

```
#include <iostream>
using namespace std;
class InsertionSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void logic()
    {
        int i, j, key;

        for (i = 1; i < n; i++)
        {
            key = arr[i];
            j = i - 1;

            while (j >= 0 && arr[j] > key)
            {
```

```
        arr[j + 1] = arr[j];
        j--;
    }

    arr[j + 1] = key;
}
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};

int main()
{
    InsertionSort is;

    is.arrayInput();

    cout << "Before Sorting: ";
    is.printArray();

    is.logic();

    cout << "After Sorting: ";
    is.printArray();

    return 0;
}
```

Output:



```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\insertionsort.exe"
Enter number of elements: 6
Enter the elements: 12
9
3
32
4
15
Before Sorting: 12 9 3 32 4 15
After Sorting: 3 4 9 12 15 32
```


Time Complexity:

Insertion sort Time complexity

```

For (i = 1; i < n; i++) {
    Key = A[i];
    j = i - 1;
    while (j >= 0 && A[j] > Key) {
        A[j+1] = A[j];
        j--;
    }
    A[j+1] = Key;
}

```

$T(n) = (n-1) + 1 + 1 + 1 + 1$
 $T(n) = n + 2$
 $T(n) = O(n)$

worst case:

$$T(n) = 1 + 2 + 3 + \dots + (n-1)$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

Average case:

$$T(n) = O(n^2)$$

4) Merge Sort:

Algorithm:

Merge Sort:

Step 1: Re Start.

Step 2: Read array A and n.

Step 3: Divide the array into two halves

Step 4: Recursively Sort then left half.

Step 5: Recursively Sort then Right half

Step 6: Repeat until the whole array is Sorted

Step 7: End.

Program:

```
#include <iostream>
using namespace std;
class MergeSort{
public:
    int arr[100],n;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void merge(int low, int mid, int high){
        int temp[100];
        int i = low;
        int j = mid + 1;
        int k = low;
        while (i <= mid && j <= high){
            if (arr[i] < arr[j]){
                temp[k] = arr[i];
                i++;
            }
            else{
                temp[k] = arr[j];
                j++;
            }
            k++;
        }
        while (i <= mid)
        {
            temp[k] = arr[i];
            i++;
            k++;
        }
        while (j <= high)
        {
            temp[k] = arr[j];
            j++;
            k++;
        }
        for (i = low; i <= high; i++)
        {
            arr[i] = temp[i];
        }
    }
    void logic(int low, int high)
    {
```

```
if (low < high)
{
    int mid = (low + high) / 2;
    logic(low, mid);
    logic(mid + 1, high);
    merge(low, mid, high);
}
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};

int main()
{
    MergeSort ms;

    ms.arrayInput();

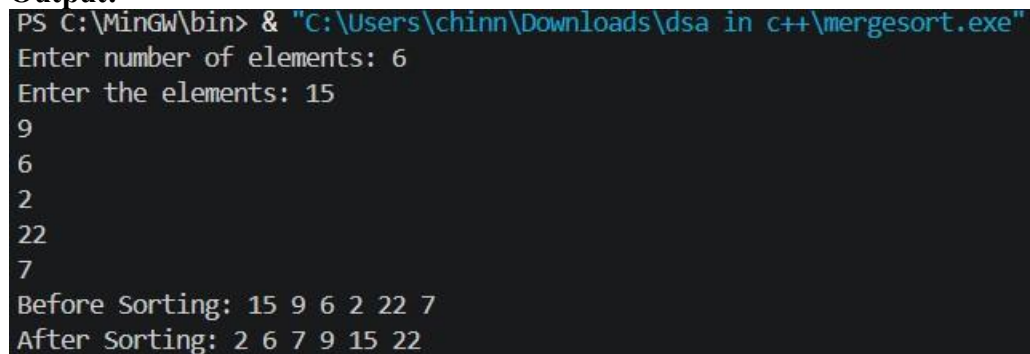
    cout << "Before Sorting: ";
    ms.printArray();

    ms.logic(0, ms.n - 1);

    cout << "After Sorting: ";
    ms.printArray();

    return 0;
}
```

Output:



```
PS C:\MingW\bin> & "C:\Users\chinn\Downloads\dsa in c++\mergesort.exe"
Enter number of elements: 6
Enter the elements: 15
9
6
2
22
7
Before Sorting: 15 9 6 2 22 7
After Sorting: 2 6 7 9 15 22
```



Time Complexity:

MergeSort (A, low, high) $\rightarrow T(n)$
 if (low < high) {
 mid = (low + high) / 2; $\rightarrow (1)$
 mergeSort (A, low, mid); $\rightarrow T(n/2)$
 mergeSort (A, mid + 1, high); $\rightarrow T(n/2)$
 mergeSort (A, low, mid, high); $\rightarrow (n)$
 }

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + n & n>1 \end{cases}$$

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2T(n/2) + n$$

$$= 2^2 T(n/4) + n + n$$

$$T(n) = 2^2 [2T(n/8) + \frac{n}{2}] + 2n$$

$$T(n) = 2^3 T(n/8) + 3n$$

$$= 2^4 T(n/16) + n + 3n$$

$$= 2^4 T(n/16) + 4n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$n = 1$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

Apply $\log$ on both side.

$$\log n = \log 2^k$$

$$\log n = k \log 2$$

$$\frac{\log n}{\log 2} = k$$

$$k = \log_2 n$$

$$T(n) = nT(n/2^k) + n \log_2 n$$

$$= n(1) + n \log_2 n$$

$$T(n) = O(n \log_2 n)$$

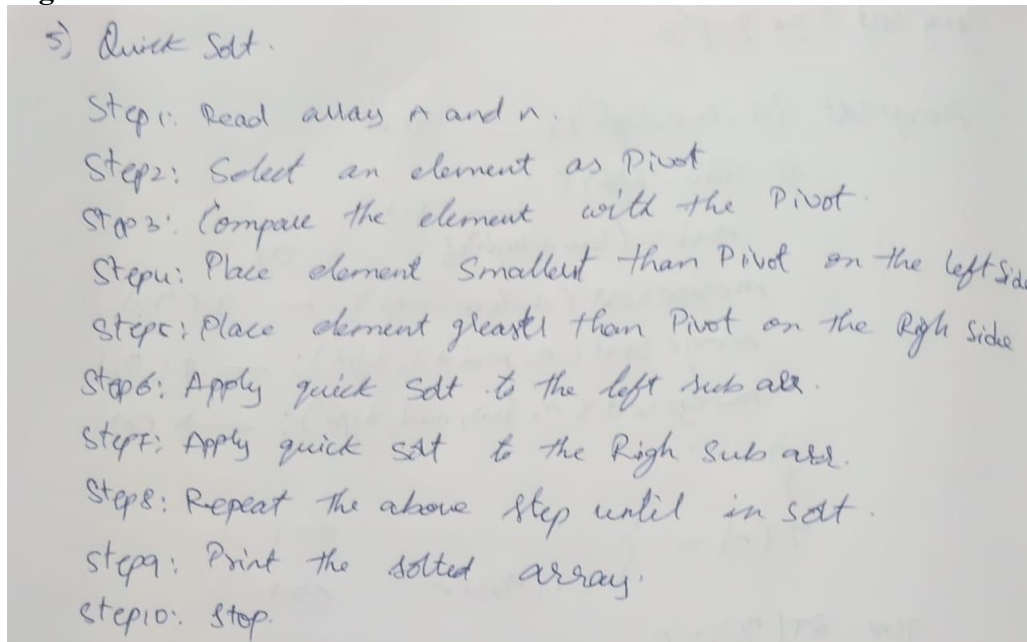
Best case: $O(n \log n)$

Worst case: $O(n \log n)$

Average case: $O(n \log n)$

5) Quick Sort:

Algorithm:



5) Quick Sort.

- Step 1: Read array n and n .
- Step 2: Select an element as Pivot.
- Step 3: Compare the element with the Pivot.
- Step 4: Place element Smaller than Pivot on the left Side.
- Step 5: Place element greater than Pivot on the Right Side.
- Step 6: Apply quick sort to the left sub arr.
- Step 7: Apply quick sort to the Right sub arr.
- Step 8: Repeat the above step until in sort.
- Step 9: Print the sorted array.
- Step 10: Stop.

Program:

```
#include <iostream>
using namespace std;
class QuickSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
    }
};
```



```
    cout << endl;
}

int partition(int arr[], int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end)
    {
        while (arr[start] <= pivot && start < ub)
        {
            start++;
        }

        while (arr[end] > pivot)
        {
            end--;
        }

        if (start < end)
        {
            swap(arr[start], arr[end]);
        }
    }

    swap(arr[lb], arr[end]);

    return end;
}

void quicksort(int arr[], int lb, int ub)
{
    if (lb < ub)
    {
        int location = partition(arr, lb, ub);

        quicksort(arr, lb, location - 1);
        quicksort(arr, location + 1, ub);
    }
}

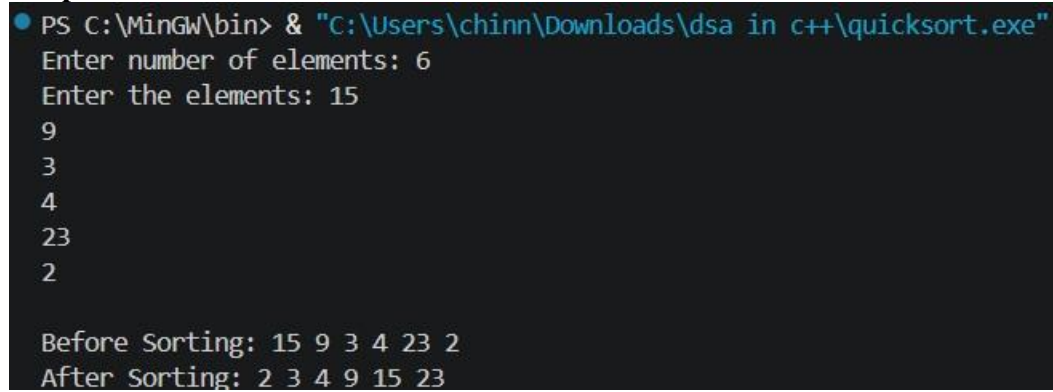
void sort()
{
    quicksort(arr, 0, n - 1);
}

};

int main()
```

```
{  
    QuickSort qs;  
  
    qs.arrayInput();  
  
    cout << "\nBefore Sorting: ";  
    qs.printArray();  
  
    qs.sort();  
  
    cout << "After Sorting: ";  
    qs.printArray();  
  
    return 0;  
}
```

Output:



```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\quicksort.exe"  
Enter number of elements: 6  
Enter the elements: 15  
9  
3  
4  
23  
2  
  
Before Sorting: 15 9 3 4 23 2  
After Sorting: 2 3 4 9 15 23
```

Time Complexity:

void quickSort(int A[], int low, int high) {
 if (low < high) {
 Partition(A, low, high);
 quickSort(A, low, P-1);
 quickSort(A, P+1, high);
 }
}

$$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + n$$

$$= 2T(\frac{n}{2}) + n$$

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(\frac{n}{2}) & n > 1 \end{cases}$$

$$T(n) = 2T(\frac{n}{2}) + n$$

$a=2, \quad b=2, \quad p>-1$
 $k=1, \quad p=0, \quad (0>-1)$

$$\log_b a = \log_2 2 = 1 = k$$

$$T(n) = O(n^k \log^{p+1} n)$$

$$= O(n \log n)$$

$$T(n) = O(n \log n)$$

Best Case: $O(n \log n)$
Worst Case: $\rightarrow n(n-1) + (n-2) + \dots + 1 \Rightarrow \frac{n(n+1)}{2} = \frac{n^2+n}{2} \Rightarrow T(n) = O(n^2)$
Average Case: $O(n \log n)$

Conclusion:

The implementation and analysis of sorting algorithms help us understand their efficiency and performance.

Merge Sort and Quick Sort are suitable for large datasets, while simpler algorithms are useful for smaller datasets.